

PROJECT PROGRESS - II

PREDICTING EPL MATCH OUTCOMES USING STATISTICAL LEARNING

**Submitted by:
ANURAG NEPAL
SAN FRANCISCO, CALIFORNIA**

**Submitted for
MATH 748
THEORY AND APPLICATIONS OF STATISTICAL MACHINE
LEARNING
SPRING 2026**

1. Description of the Problem

Sports analytics has transformed from a niche interest into a multibillion-dollar industry used by professional clubs and betting markets. However, accurately predicting the outcome of a football (soccer) match is notoriously difficult due to the low-scoring nature of the game and the high degree of randomness.

The goal of this project is to develop a statistically sound machine learning framework to predict the outcome of English Premier League (EPL) matches (Home Win, Draw, Away Win). Rather than focusing solely on classification accuracy, this project aims to estimate well-calibrated probabilities. This approach allows for a deeper understanding of the stochastic processes driving match results, such as team attack strength and defense strength.

2. Dataset Description

The project will use the [Football-Data.co.uk](https://www.football-data.co.uk) repository, specifically the English Premier League match data covering the last 5 completed seasons. This dataset is publicly available and machine-readable. It includes the following key features:

Match Outcomes

- FTR: Full Time Result (H (Home Win), D(Draw), A (Away Win))
- FTHG / FTAG: Full Time Home Goals and Away Goals

Predictive Signals (Performance Metrics)

- HS / AS: Home Team Shots / Away Team Shots
- HST / AST: Home/Away Shots on Target
- HC / AC: Home/Away Corners
- HF/AF: Home/Away Fouls.

Betting Odds (Benchmark)

- B365H, B365D, B365A: Odds from Bet365, serving as an information-efficient baseline for model evaluation for games yet to happen.

Table 1 Variable Statistics Summary

	FTHG	FTAG	HS	AS	HST	AST	HC	AC	HF	AF
mean	1.80	1.48	15.40	12.18	5.44	4.43	6.12	4.71	10.74	11.35
std	1.37	1.28	6.26	5.47	2.89	2.53	3.22	2.91	3.46	3.67
min	0.00	0.00	1.00	2.00	0.00	0.00	0.00	0.00	2.00	2.00
max	6.00	8.00	36.00	30.00	15.00	15.00	17.00	15.00	20.00	22.00

From the initial statistical summary prior to visualization, home teams scored an average of 1.80 goals per game (with a max of 6), while Away teams scored an average of 1.48 goals (with a max of 8). Home teams also averaged a higher volume of offensive output, registering more shots (15.40 vs. 12.18) and shots on target (5.44 vs. 4.43) than their visiting opponents.

3. Data Cleaning

The Football-Data.co.uk CSV files are pre-formatted for machine readability and modelling. A preliminary check for missing values across the dataset confirmed that there are exactly 0 missing values (NA) in the selected columns for the 23/24 season. The string Date column was converted into standardized datetime objects and irrelevant auxiliary betting market columns other than Bet365 odds were dropped to streamline the analysis. The raw decimal odds (B365H, B365D, B365A) were converted into implied probabilities and normalized to account for the bookmaker's profit margin.

4. Data Visualizations and Findings

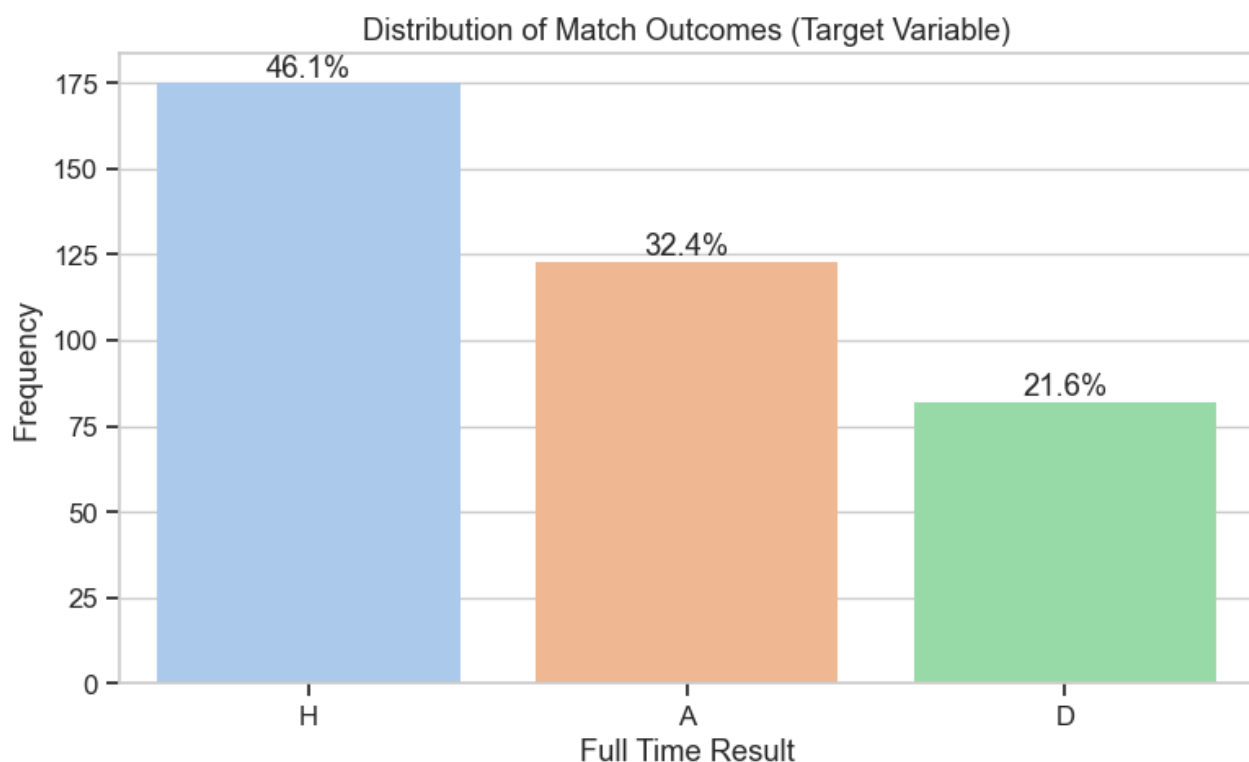


Figure 1 Target Variable Distribution

The count plot of the target variable, Full Time Results (FTR), visually confirms the well-documented phenomenon of "Home Advantage" in football. Home teams win nearly half of all matches (46.1%), while Away Wins account for 32.4%, and Draws occur in 21.6% of the matches. So, it becomes apparent that the teams have an advantage playing at home compared to away grounds. This could be because of the supporter numbers or just the teams being used to play at their own ground where they can train. A naive baseline classifier that simply guesses "Home Win" for every single match would achieve 46.1% accuracy, so that the models must exceed this threshold to be considered useful.

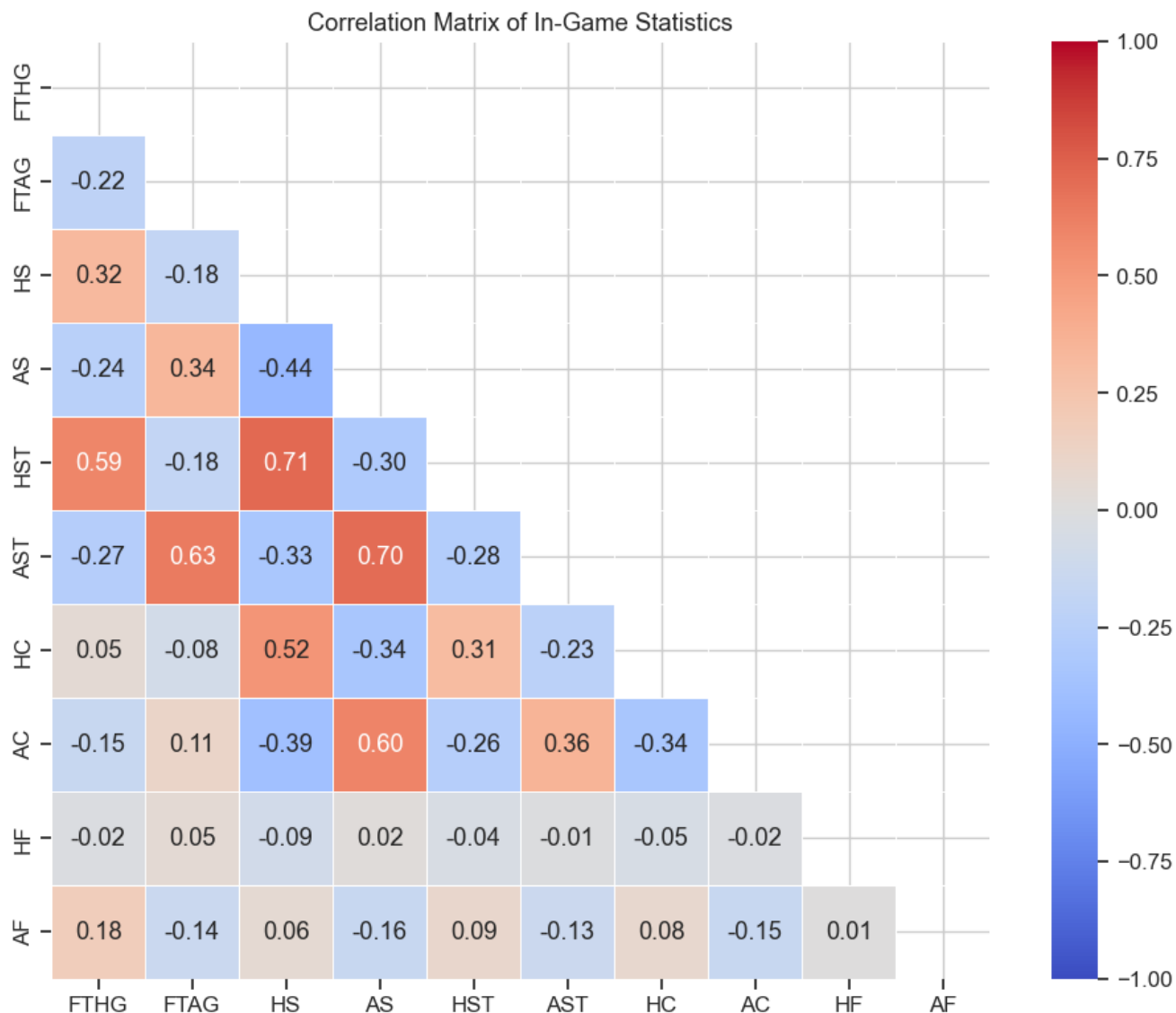


Figure 2 Correlation Matrix

The correlation heatmap reveals that there is intense multicollinearity among the offensive features, which will artificially inflate variance if they are left unaddressed. There is a strong positive correlation ($r = 0.71$) between Home Shots (HS) and Home Shots on Target (HST). Similarly, Away Shots and Away Shots on Target are highly correlated ($r = 0.75$). Furthermore, Home Goals (FTHG) show a strong correlation with Home Shots on Target ($r = 0.63$). This mathematical relationship tells that l_1 (Lasso) or l_2 (Ridge) regularization will be necessary when building linear predictive models to control model complexity and feature weight instability.

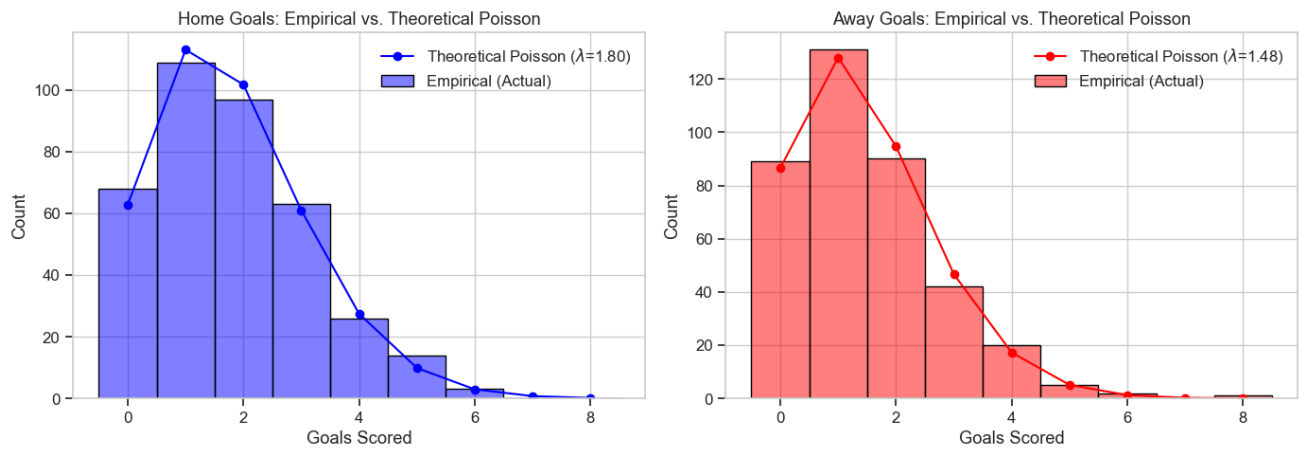


Figure 3 Poisson Distribution Verification

A Double Poisson regression model is a standard approach in sports modeling. To justify this, the empirical goal distributions for both home and away teams were plotted against a theoretical Poisson probability mass function. The Home Goals empirically follow a Poisson curve with a mean of $\lambda=1.80$, while Away Goals follow a curve with $\lambda=1.48$. The exceptionally close fit validates the standard approach that goal-scoring in football can be modeled as independent Poisson processes.

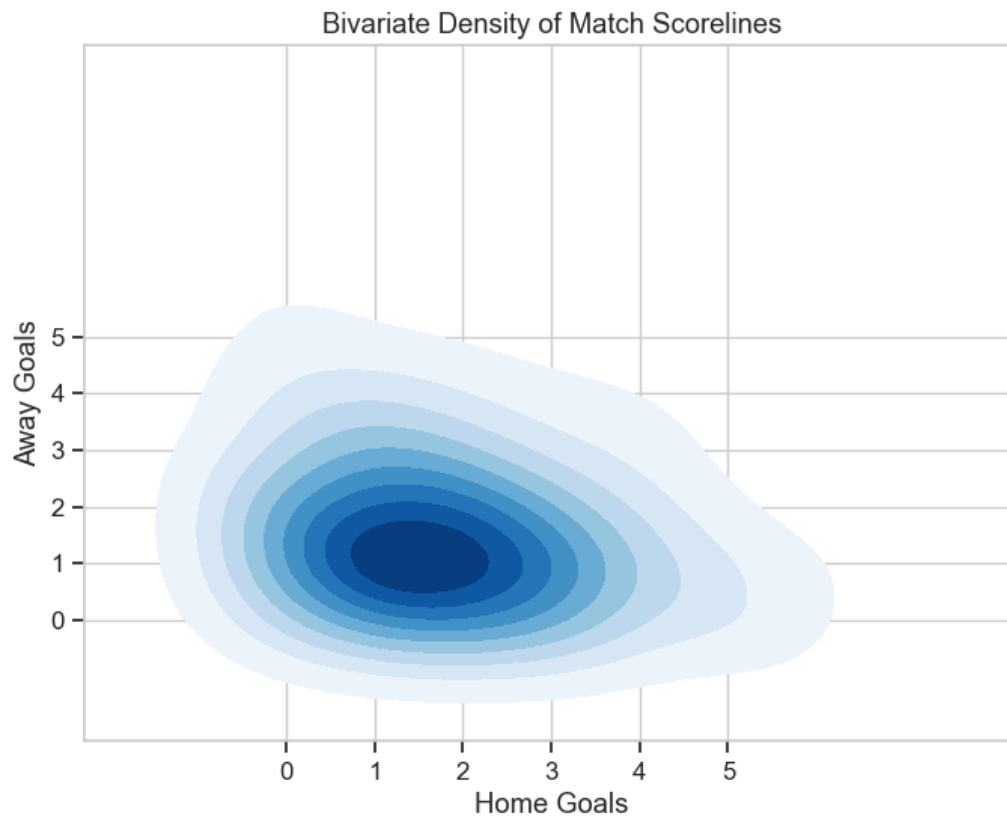


Figure 4 Uncertainty in Scorelines

The bivariate density contour plot constructed to visualize the joint distribution of goals by mapping Home Goals against Away Goals. The contour plot shows a tight, concentrated density mass around low-scoring outcomes (specifically 1-1, 1-0, and 2-1). The wide, rapidly scattering outer bands demonstrate the high variance and uncertainty present in the dataset, and it visually proves why precise score, prediction is highly volatile and will be a challenge to correctly predict. There can never be enough data to correctly predict the scoreline as the game is very fluid and the game state defines the result. And with the advent of VAR (Video Assisted Refereeing), there is even more scrutiny when a goal is scored.

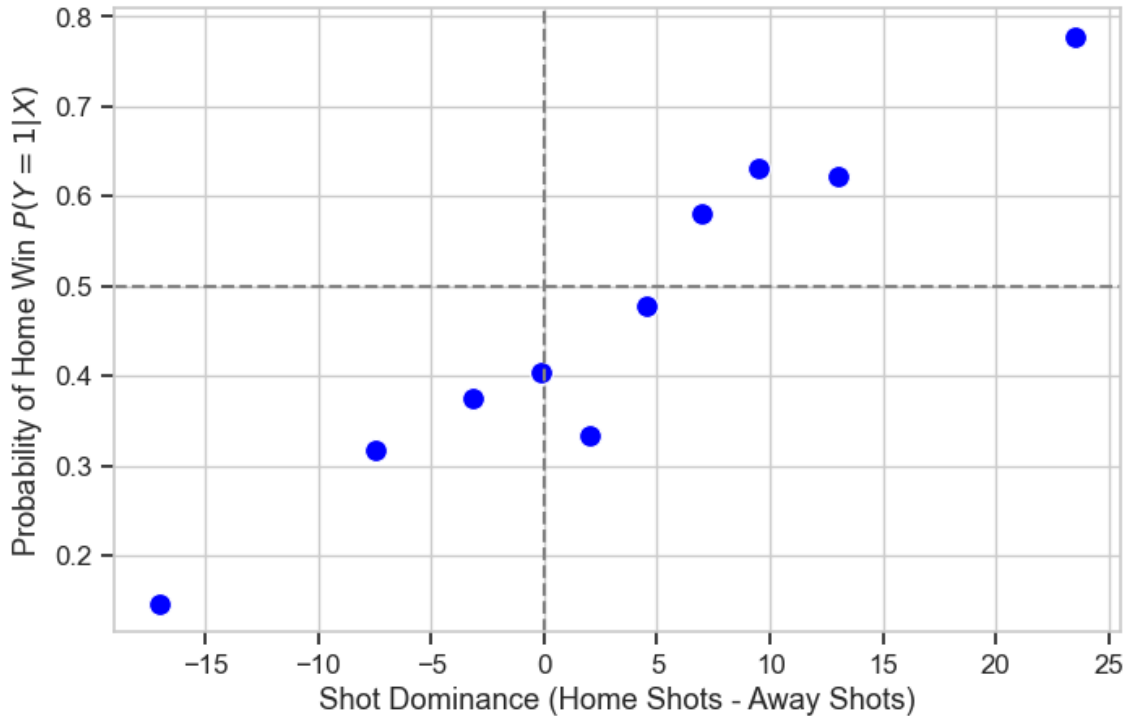


Figure 5 Win Probability Vs Shots taken

To test if a logistic function is appropriate for a discriminative classification model, "shot dominance" (Home Shots minus Away Shots) was calculated and plotted against the empirical probability of a Home Win. The resulting scatter plot scales smoothly from roughly 0.2 probability at negative dominance up to nearly 0.8 probability at high positive dominance. This shape provides enough evidence that the linear combination of features can be mapped through a logistic function $\sigma(z) = \frac{1}{1+e^{-z}}$.

To establish a rigorous and information-efficient baseline, the calibration of commercial betting odds (Bet365) was evaluated. The probabilities were grouped into bins to plot the mean predicted probability against the fraction of actual outcomes.

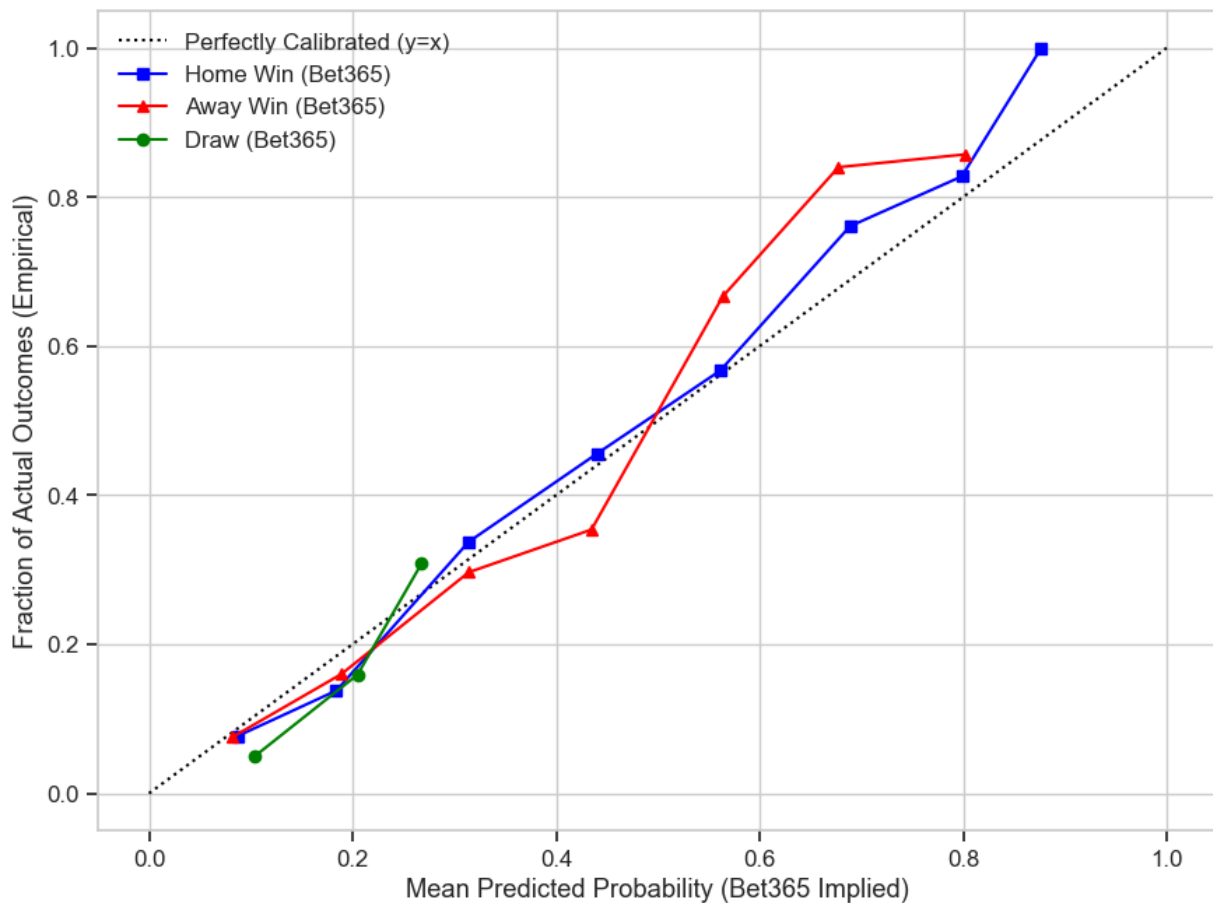


Figure 6 Betting Market Calibration

The diagram shows that the Bet365 betting market is very well-calibrated. Both the Home Win and Away Win curves tightly track the perfectly calibrated $y=x$ reference line across the entire probability spectrum. This proof of market efficiency tells me that evaluating the machine learning models on raw classification accuracy alone is insufficient. It must be done with proper scoring rules, to ensure the models generate probability distributions capable of getting close to this commercial baseline.

5. Feature Engineering and Temporal Dynamics

As identified during the exploratory data analysis, team performance is highly volatile and non-stationary. To transition from exploratory analysis to a predictive framework, raw match statistics were transformed into lagged features that capture team momentum while strictly preventing data leakage.

Leakage Prevention and Engineered Features

To capture these dynamic effects without introducing look-ahead bias (data leakage), several lagged predictive signals were engineered from the raw match logs. Every rolling and expanding window was strictly shifted by one match (shift=1) prior to aggregation, ensuring that the model only learns from historical data available immediately before kickoff.

For early-season matches where a 5-game history was unavailable, expanding averages were utilized to allow the model to learn from available early-season fixtures rather than dropping rows.

A set of dynamic predictors was constructed to represent the "state" of each team entering a match:

- **Rolling Form (5-Game Window) and League Momentum:** A team's raw attacking and defensive strength fluctuates throughout the season. To capture this momentum, 5-game rolling averages were constructed for Goals For (GF), Goals Against (GA), and Shots on Target (SOT). To quantify psychological momentum and overall form, a "League Momentum" feature was engineered by calculating a rolling 5-game sum of match points earned (W=3, D=1, L=0). To prevent the model from discarding early-season fixtures where 5 prior games are unavailable, a minimum period of 1 (min_periods=1) was utilized, defaulting truly first-game rows to zero. This allows the model to continuously learn as early-season form stabilizes.
- **Venue-Specific Form:** General form often masks a team's true capability at their home stadium versus on the road. To decouple these effects, venue-specific expanding averages were calculated. For the Home team, an expanding mean of goals and SOT was calculated strictly using prior matches played at their home venue (the "fortress" effect). Conversely, the Away team's form was tracked using an expanding mean of their prior away fixtures (the "road warrior" effect).
- **Geographical Fatigue:** Travel fatigue is a well-documented physical and psychological factor in professional sports. To quantify this, public stadium GPS coordinates (latitude and longitude) for all 20 Premier League clubs were mapped. The Haversine formula was utilized to calculate the great-circle distance (in miles) between stadiums. By tracking each team's journey leg-by-leg, a cumulative 5-game rolling travel distance was computed. This metric serves as a proxy for accumulated geographical fatigue.

Table 2 Summary of Engineered Features

	H_Ro ll_GF _5	H_Rol l_SOT _5	H_Home _GF_For m	H_Home _SOT_Fo rm	H_Rol l_Pts_ 5	H_Travel _Fatigue _5	A_Ro ll_GF _5	A_Rol l_SOT _5	A_Away _GF_For m	A_Away _SOT_Fo rm	A_Rol l_Pts_ _5	A_Travel _Fatigue _5
co un t	380.0	380.0	380.0	380.0	380.0	380.0	380.0	380.0	380.0	380.0	380.0	380.0

m	1.558	4.718	1.633	4.932	6.308	471.613	1.602	4.852	1.342	4.124	6.492	444.86
ea												
n												
st	0.773	1.73	0.846	2.047	3.797	241.32	0.796	1.783	0.628	1.674	3.768	233.919
d												
mi	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
n												
25	1.0	3.6	1.062	3.889	4.0	334.909	1.0	3.8	1.0	3.321	4.0	314.311
%												
50	1.5	4.6	1.577	4.833	6.0	459.619	1.6	4.8	1.333	4.185	6.0	438.244
%												
75	2.0	5.8	2.191	6.354	9.0	619.608	2.0	5.8	1.714	4.86	10.0	563.501
%												
m	4.8	10.0	5.0	13.0	15.0	1165.014	5.0	13.0	4.0	10.667	15.0	1056.823
ax												

The summary statistics of the engineered matrix (across all 380 matches) confirm the successful calculation of our signals. The average 5-game rolling points for home teams sits at 6.308, indicating an average baseline form across the league.

Table 3 Venue Specific Deltas

HomeTeam	Home GF minus Away GF
Aston Villa	1.59
Liverpool	0.838
Newcastle	0.777
Arsenal	0.556
Nott'm Forest	0.392
Man United	0.368
Fulham	0.339
Brentford	0.294
Man City	0.17
Brighton	0.155
Luton	0.149
Wolves	0.139
West Ham	0.126
Sheffield United	0.113
Crystal Palace	0.087
Chelsea	0.043
Burnley	0.02
Everton	-0.325
Tottenham	-0.346
Bournemouth	-0.464

Analyzing the venue-specific form deltas reveals that certain teams are highly dependent on their home grounds. For instance, Aston Villa exhibits a massive +1.590 difference between their average home goals scored and away goals scored, solidifying their "fortress" status. Conversely, teams like Bournemouth (-0.464) and Tottenham (-0.346) actually performed slightly better offensively on the road.

Table 4 Geographical Fatigue

Date	HomeTeam	AwayTeam	H Travel Fatigue 5
2023-12-02	Newcastle	Man United	1165.014386825000
2024-04-02	Newcastle	Everton	1128.0750686681300
2023-11-25	Newcastle	Chelsea	1118.446198170030
2024-05-11	Newcastle	Brighton	1093.3797827305400
2024-03-13	Bournemouth	Luton	1051.8756028563000

Also, the geographical fatigue metric captured significant logistical disadvantages. Newcastle United experienced the highest physical toll, registering up to 1,165 miles of cumulative travel over a 5-game stretch prior to their home match against Manchester United in December 2023.

6. Baseline Logistic Regression Model

With the multicollinearity and logistic relationships identified during the EDA, a regularized Multinomial Logistic Regression model was established as the baseline predictive framework. Logistic Regression is highly interpretable, outputs the continuous probability distributions required for this problem space and serves as a robust benchmark before transitioning to complex non-linear models.

Standard k-fold cross-validation is mathematically inappropriate for sports data, as it randomizes the dataset and allows future matches to predict past matches, violating causality. To simulate a true "live" betting environment, a Walk-Forward Time-Series Cross-Validation approach was implemented using 5 temporal splits. The model trains on a chronologically expanding window of past matches and is evaluated strictly on the subsequent block of future matches.

Due to the intense multicollinearity observed between offensive metrics (e.g., Shots and Shots on Target), standardizing the feature space was a strict prerequisite. A StandardScaler was applied within each fold to prevent data leakage from the test set. The Logistic Regression model was trained utilizing the lbfgs solver and a moderate L_2 regularization penalty ($C=0.1$) to shrink unstable coefficient weights and prevent overfitting on highly correlated inputs.

As proven by the Bet365 market calibration during the EDA, raw accuracy is an insufficient metric for football predictions. The model was evaluated using Logarithmic Loss (Log-Loss), a proper scoring rule that heavily penalizes confident but incorrect probability estimations.

$$LogLoss = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \log(p_{ij})$$

The Cross-Validated Log-Loss of the baseline Logistic Regression model was **1.0258**. When compared against the highly efficient Bet365 commercial market baseline, which achieved a Log-Loss of **0.9092** after overround normalization, the baseline model performs reasonably

well but still lags behind the sportsbooks. The market's lower Log-Loss indicates that bookmakers are integrating external information (e.g., injuries, manager changes, expected goals (xG), and line-up data) that our current purely historical-stat-driven model cannot capture.

Table 5 Logistic Regression Coefficients

	Coefficient
H_Roll_SOT_5	0.22796583067346900
H_Roll_Pts_5	0.1944118667508610
H_Travel_Fatigue_5	0.09005013150963370
H_Home_GF_Form	0.07995295272555420
A_Roll_GF_5	0.07873496043458830
H_Home_SOT_Form	0.03324247994541560
A_Travel_Fatigue_5	0.023768159530395600
H_Roll_GF_5	-0.02342668240211980
A_Away_GF_Form	-0.03721283694005760
A_Roll_SOT_5	-0.13350371672212200
A_Roll_Pts_5	-0.16091689333349700
A_Away_SOT_Form	-0.1679751927231610

Analyzing the learned coefficients provides strong statistical intuition into the drivers of a Home Win. The most significant positive driver is the Home team's recent offensive accuracy (H_Roll_SOT_5 with a weight of +0.228) and their overall League Momentum (H_Roll_Pts_5 at +0.194). Interestingly, the Home team's general goals scored feature (H_Roll_GF_5) received a slightly negative coefficient (-0.023); this is a classic manifestation of the $\$L_2\$$ regularization handling multicollinearity, effectively telling us that *Shots on Target* is a much more stable and reliable predictive signal than raw goals scored, which contains more variance and luck. Conversely, the strongest factors preventing a Home Win are the Away team's venue-specific defensive/offensive rigidity (A_Away_SOT_Form at -0.168) and the Away team's overall league momentum (A_Roll_Pts_5 at -0.161).

7. Conclusion and Future Work

The successful implementation of non-leaking temporal features and a time-series validated baseline model proves that match outcomes can be mathematically modeled using historical match data. While the regularized Logistic Regression framework correctly identified key drivers like rolling Shots on Target and venue-specific form, its Log-Loss (1.0258) indicates that a simple linear combination of features is not fully capturing the complex, non-linear interactions within a football match.

The next phase of the project will focus on Implementing tree-based ensemble methods, specifically XGBoost or Random Forests, to capture non-linear feature interactions (e.g., how Away Travel Fatigue interacts specifically with a High Home SOT Form) and utilizing grid search over the temporal folds to optimize the learning rate, tree depth, and regularization parameters of the advanced models to close the gap with the commercial betting market baseline.

APPENDIX

```
!pip3 install pandas numpy matplotlib seaborn scikit-learn statsmodels
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import poisson
from sklearn.calibration import calibration_curve

plt.style.use('seaborn-v0_8-whitegrid')
sns.set_context("notebook", font_scale=1.1)
# =====
# 1. LOAD THE DATA
# =====
url = "https://www.football-data.co.uk/mmz4281/2324/E0.csv"
df = pd.read_csv(url)
df
# Select only the relevant statistical columns to keep the EDA focused
cols_to_keep = ['Date', 'HomeTeam', 'AwayTeam', 'FTHG', 'FTAG', 'FTR',
                'HS', 'AS', 'HST', 'AST', 'HC', 'AC', 'HF', 'AF',
                'B365H', 'B365D', 'B365A']
df = df[cols_to_keep].copy()
# =====
# 2. SUMMARY STATISTICS
# =====
print("--- DATASET OVERVIEW ---")
print(f"Total Matches: {len(df)}")
print("\n--- MISSING VALUES ---")
print(df.isnull().sum().max() == 0) # Check if there are any missing values
print("\n--- CONTINUOUS VARIABLE STATISTICS ---")
# Focus on mean, std, and max for goals and shots
stat_cols = ['FTHG', 'FTAG', 'HS', 'AS', 'HST', 'AST', 'HC', 'AC', 'HF', 'AF']
print(df[stat_cols].describe().round(2).loc[['mean', 'std', 'min', 'max']])
# =====
# 3. PLOT: TARGET VARIABLE DISTRIBUTION
# =====
plt.figure(figsize=(8, 5))
# FTR: H = Home Win, D = Draw, A = Away Win
ax = sns.countplot(data=df, x='FTR', order=['H', 'A', 'D'], palette='pastel')
plt.title('Distribution of Match Outcomes (Target Variable)')
plt.ylabel('Frequency')
plt.xlabel('Full Time Result')
# Add percentage labels to check baseline accuracy
total = len(df)
```

```

for p in ax.patches:
    percentage = f'{100 * p.get_height() / total:.1f}%'
    x = p.get_x() + p.get_width() / 2
    y = p.get_height()
    ax.annotate(percentage, (x, y), ha='center', va='bottom')

plt.tight_layout()
plt.show()
# =====
# 4. PLOT: MULTICOLLINEARITY CHECK (CORRELATION MATRIX)
# =====
# In GLMs, highly correlated features (e.g., Shots and Shots on Target) inflate variance.
plt.figure(figsize=(10, 8))
corr_matrix = df[['FTHG', 'FTAG', 'HS', 'AS', 'HST', 'AST', 'HC', 'AC', 'HF',
'AF']].corr()

# Mask the upper triangle for a cleaner look
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

sns.heatmap(corr_matrix, mask=mask, annot=True, fmt=".2f", cmap='coolwarm',
            vmin=-1, vmax=1, square=True, linewidths=.5)
plt.title('Correlation Matrix of In-Game Statistics')
plt.tight_layout()
plt.show()
# =====
# 4. POISSON DISTRIBUTION CHECK
# =====
home_goals = df['FTHG']
away_goals = df['FTAG']

# Calculate Lambda (mean) for home and away
lambda_home = home_goals.mean()
lambda_away = away_goals.mean()

# Generate theoretical Poisson probabilities
max_goals = 8
x_vals = np.arange(0, max_goals + 1)
poisson_home = poisson.pmf(x_vals, lambda_home) * len(df)
poisson_away = poisson.pmf(x_vals, lambda_away) * len(df)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Home Goals fit
sns.histplot(home_goals, bins=np.arange(0, max_goals+2)-0.5, stat='count', alpha=0.5,
ax=axes[0], color='blue', label='Empirical (Actual)')

```

```

        axes[0].plot(x_vals, poisson_home, 'bo-', label=f'Theoretical Poisson
($\lambda$={lambda_home:.2f})')
        axes[0].set_title('Home Goals: Empirical vs. Theoretical Poisson')
        axes[0].set_xlabel('Goals Scored')
        axes[0].legend()

# Away Goals fit
sns.histplot(away_goals, bins=np.arange(0, max_goals+2)-0.5, stat='count', alpha=0.5,
ax=axes[1], color='red', label='Empirical (Actual)')
        axes[1].plot(x_vals, poisson_away, 'ro-', label=f'Theoretical Poisson
($\lambda$={lambda_away:.2f})')
        axes[1].set_title('Away Goals: Empirical vs. Theoretical Poisson')
        axes[1].set_xlabel('Goals Scored')
        axes[1].legend()

plt.tight_layout()
plt.show()
# =====
# 5. BIVARIATE JOINT DISTRIBUTION OF SCORELINES
# =====
# Shows the aleatoric uncertainty and most probable outcomes
plt.figure(figsize=(8, 6))
sns.kdeplot(x=df['FTHG'], y=df['FTAG'], cmap="Blues", fill=True, bw_adjust=1.5,
thresh=0.05)
plt.title('Bivariate Density of Match Scorelines')
plt.xlabel('Home Goals')
plt.ylabel('Away Goals')
plt.xticks(np.arange(0, 6, 1))
plt.yticks(np.arange(0, 6, 1))
plt.show()
# =====
# 6. EMPIRICAL LOGISTIC CURVE (Shot Dominance vs Win Prob)
# =====
# Calculates Home Shot Difference and the empirical probability of a Home Win
df['Shot_Diff'] = df['HS'] - df['AS']
df['Home_Win'] = (df['FTR'] == 'H').astype(int)

# Group by shot difference bins to find empirical probability
bins = pd.qcut(df['Shot_Diff'], q=10, duplicates='drop')
prob_df = df.groupby(bins)['Home_Win'].mean().reset_index()
prob_df['Bin_Center'] = prob_df['Shot_Diff'].apply(lambda x: x.mid).astype(float)

plt.figure(figsize=(8, 5))
# Plotting just the empirical data points without the statsmodels logistic fit
sns.scatterplot(x='Bin_Center', y='Home_Win', data=prob_df, s=100, color='blue')
plt.title('Empirical Win Probability vs. Shot Difference')

```

```

plt.xlabel('Shot Dominance (Home Shots - Away Shots)')
plt.ylabel('Probability of Home Win  $P(Y=1 | X)$ ')
plt.axhline(0.5, color='gray', linestyle='--')
plt.axvline(0, color='gray', linestyle='--')
plt.show()
# =====
# 7. CALCULATE IMPLIED PROBABILITIES & CALCULATE CALIBRATION
CURVES
# =====
# Decimal odds represent inverse probability.
# Example: Odds of 2.00 =  $1/2.00 = 0.50$  (50% chance)
df['Implied_H'] = 1 / df['B365H']
df['Implied_D'] = 1 / df['B365D']
df['Implied_A'] = 1 / df['B365A']

# Bookmakers include a profit margin ("vig" or "overround"), so probabilities sum to
> 1.0
# We must divide by the sum to normalize them into true probabilities (summing to
1.0)
df['Overround'] = df['Implied_H'] + df['Implied_D'] + df['Implied_A']
df['Prob_H'] = df['Implied_H'] / df['Overround']
df['Prob_D'] = df['Implied_D'] / df['Overround']
df['Prob_A'] = df['Implied_A'] / df['Overround']

# Create binary targets for scikit-learn's calibration_curve
df['Actual_H'] = (df['FTR'] == 'H').astype(int)
df['Actual_D'] = (df['FTR'] == 'D').astype(int)
df['Actual_A'] = (df['FTR'] == 'A').astype(int)

# Bins the predicted probabilities and calculates the true fraction of positives in each
bin
frac_pos_H, mean_pred_H = calibration_curve(df['Actual_H'], df['Prob_H'],
n_bins=8, strategy='uniform')
frac_pos_A, mean_pred_A = calibration_curve(df['Actual_A'], df['Prob_A'],
n_bins=8, strategy='uniform')
frac_pos_D, mean_pred_D = calibration_curve(df['Actual_D'], df['Prob_D'],
n_bins=8, strategy='uniform')

plt.figure(figsize=(9, 7))

# Plot the perfectly calibrated reference line (y = x)
plt.plot([0, 1], [0, 1], "k:", label="Perfectly Calibrated (y=x)")

# Plot the market's empirical calibration
plt.plot(mean_pred_H, frac_pos_H, "s-", color='blue', label="Home Win (Bet365)")
plt.plot(mean_pred_A, frac_pos_A, "^-", color='red', label="Away Win (Bet365)")

```

```

plt.plot(mean_pred_D, frac_pos_D, "o-", color='green', label="Draw (Bet365)")

plt.title('Reliability Diagram: Betting Market Calibration')
plt.xlabel('Mean Predicted Probability (Bet365 Implied)')
plt.ylabel('Fraction of Actual Outcomes (Empirical)')
plt.legend(loc='upper left')
plt.xlim([-0.05, 1.05])
plt.ylim([-0.05, 1.05])
plt.grid(True)
plt.tight_layout()
plt.show()
# =====
# 5. MODULAR TEAM EDA (e.g., MANCHESTER CITY)
# =====
TARGET_TEAM = 'Man City'

# Filter data where target team played
team_matches = df[(df['HomeTeam'] == TARGET_TEAM) | (df['AwayTeam'] ==
TARGET_TEAM)].copy()

# Calculate goals scored and conceded for the target team
team_matches['Team_Scored'] = np.where(team_matches['HomeTeam'] ==
TARGET_TEAM, team_matches['FTHG'], team_matches['FTAG'])
team_matches['Team_Conceded'] = np.where(team_matches['HomeTeam'] ==
TARGET_TEAM, team_matches['FTAG'], team_matches['FTHG'])
team_matches['Opponent'] = np.where(team_matches['HomeTeam'] ==
TARGET_TEAM, team_matches['AwayTeam'], team_matches['HomeTeam'])

# Plot: Goals Scored vs Conceded distribution for the specific team
plt.figure(figsize=(10, 5))
sns.kdeplot(data=team_matches['Team_Scored'], fill=True, label='Goals Scored',
color='blue', alpha=0.5)
sns.kdeplot(data=team_matches['Team_Conceded'], fill=True, label='Goals
Conceded', color='red', alpha=0.5)
plt.title(f'{TARGET_TEAM}: Distribution of Goals Scored vs. Conceded')
plt.xlabel('Goals per Match')
plt.ylabel('Density')
plt.legend()
plt.tight_layout()
plt.show()
# =====
# 6. MODULAR TEAM NON-STATIONARITY (Form over Time)
# =====
TARGET_TEAM = 'Man City'

```

```

team_matches = df[(df['HomeTeam'] == TARGET_TEAM) | (df['AwayTeam'] ==
TARGET_TEAM)].copy()
team_matches['Team_Shots'] = np.where(team_matches['HomeTeam'] ==
TARGET_TEAM, team_matches['HS'], team_matches['AS'])
team_matches['Opp_Shots'] = np.where(team_matches['HomeTeam'] ==
TARGET_TEAM, team_matches['AS'], team_matches['HS'])

# Calculate 5-game rolling averages to show variance over a season
team_matches['Rolling_Shots'] = team_matches['Team_Shots'].rolling(window=5,
min_periods=1).mean()
team_matches['Rolling_Opp_Shots'] =
team_matches['Opp_Shots'].rolling(window=5, min_periods=1).mean()
team_matches['Match_Number'] = range(1, len(team_matches) + 1)

plt.figure(figsize=(10, 5))
plt.plot(team_matches['Match_Number'], team_matches['Rolling_Shots'],
label=f'{TARGET_TEAM} Shots (5-game avg)', color='skyblue', linewidth=2)
plt.plot(team_matches['Match_Number'], team_matches['Rolling_Opp_Shots'],
label='Opponent Shots (5-game avg)', color='salmon', linewidth=2)
plt.fill_between(team_matches['Match_Number'], team_matches['Rolling_Shots'],
team_matches['Rolling_Opp_Shots'], alpha=0.2, color='gray')

plt.title(f'{TARGET_TEAM}: Attacking vs. Defensive Volatility Over the Season')
plt.xlabel('Matchway (Chronological)')
plt.ylabel('Number of Shots')
plt.legend()
plt.tight_layout()
plt.show()

import pandas as pd
import numpy as np
from math import radians, sin, cos, sqrt, atan2

# 1. STADIUM GPS COORDINATES (latitude, longitude)

STADIUM_COORDS = {
    "Arsenal": (51.5549, -0.1084), # Emirates Stadium
    "Aston Villa": (52.5092, -1.8847), # Villa Park
    "Bournemouth": (50.7352, -1.8380), # Vitality Stadium
    "Brentford": (51.4907, -0.2884), # Brentford Community Stadium
    "Brighton": (50.8618, -0.0861), # Amex Stadium
    "Burnley": (53.7889, -2.2298), # Turf Moor
    "Chelsea": (51.4817, -0.1910), # Stamford Bridge
    "Crystal Palace": (51.3983, -0.0855), # Selhurst Park
    "Everton": (53.4388, -2.9664), # Goodison Park

```

```

"Fulham":      (51.4749, -0.2217), # Craven Cottage
"Liverpool":   (53.4308, -2.9608), # Anfield
"Luton":       (51.8836, -0.4318), # Kenilworth Road
"Man City":    (53.4831, -2.2004), # Etihad Stadium
"Man United":  (53.4631, -2.2913), # Old Trafford
"Newcastle":   (54.9756, -1.6218), # St James' Park
"Nott'm Forest": (52.9399, -1.1327), # City Ground
"Sheffield United": (53.3703, -1.4705), # Bramall Lane
"Tottenham":   (51.6043, -0.0665), # Tottenham Hotspur Stadium
"West Ham":    (51.5386, -0.0163), # London Stadium
"Wolves":      (52.5904, -2.1302), # Molineux Stadium
}

```

2. HAVERSINE FORMULA (returns distance in miles)

```
EARTH_RADIUS_MILES = 3958.8
```

```

def haversine_miles(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
    """Great-circle distance between two GPS points in miles."""
    lat1, lon1, lat2, lon2 = map(radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = sin(dlat / 2) ** 2 + cos(lat1) * cos(lat2) * sin(dlon / 2) ** 2
    return EARTH_RADIUS_MILES * 2 * atan2(sqrt(a), sqrt(1 - a))

```

3. BUILD PER-TEAM MATCH LOG

```

# Reshape from wide (one row per match) to long (one row per team per match)
# so that rolling operations can be applied cleanly per-team.

```

```

home_rows = df[["Date", "HomeTeam", "AwayTeam", "FTHG", "FTAG", "FTR",
                "HST", "AST"]].copy()
home_rows.columns = ["Date", "Team", "Opponent", "GF", "GA", "FTR", "SOT_F",
"SOT_A"]
home_rows["Venue"] = "H"

away_rows = df[["Date", "AwayTeam", "HomeTeam", "FTAG", "FTHG", "FTR",
                "AST", "HST"]].copy()
away_rows.columns = ["Date", "Team", "Opponent", "GF", "GA", "FTR", "SOT_F",
"SOT_A"]
away_rows["Venue"] = "A"

team_log = (
    pd.concat([home_rows, away_rows], ignore_index=True)
    .sort_values(["Team", "Date"])

```

```

        .reset_index(drop=True)
    )

    def points_earned(row) -> int:
        if row["Venue"] == "H":
            return 3 if row["FTR"] == "H" else (1 if row["FTR"] == "D" else 0)
        return 3 if row["FTR"] == "A" else (1 if row["FTR"] == "D" else 0)

    team_log["Points_Earned"] = team_log.apply(points_earned, axis=1)

```

4. ROLLING HELPERS

WINDOW = 5

```

def roll_mean(s: pd.Series) -> pd.Series:
    """Leakage-free rolling mean: shift by 1, then roll over prior 5 games."""
    return s.shift(1).rolling(window=WINDOW, min_periods=1).mean().fillna(0)

def roll_sum(s: pd.Series) -> pd.Series:
    """Leakage-free rolling sum: shift by 1, then sum prior 5 games."""
    return s.shift(1).rolling(window=WINDOW, min_periods=1).sum().fillna(0)

```

5. FEATURE 1 — OVERALL ROLLING FORM (Goals, Shots on Target)

```

grp = team_log.groupby("Team")

team_log["Roll_GF_5"] = grp["GF"].transform(roll_mean) # Avg goals scored
team_log["Roll_GA_5"] = grp["GA"].transform(roll_mean) # Avg goals conceded
team_log["Roll_SOT_5"] = grp["SOT_F"].transform(roll_mean) # Avg shots on target

```

6. FEATURE 2 — LEAGUE MOMENTUM (rolling 5-game points)

```

team_log["Roll_Pts_5"] = grp["Points_Earned"].transform(roll_sum)

```

7. FEATURE 3 — VENUE-SPECIFIC FORM

```

def venue_expanding_mean(group_df: pd.DataFrame, venue_flag: str, col: str) ->
pd.Series:
    """
    Expanding mean restricted to a single venue type, shifted by 1.
    NaN for the opposite venue type is masked; 0 before the first such game.
    """

```

```

mask = group_df["Venue"] == venue_flag
return (
    group_df[col]
    .where(mask)
    .expanding()
    .mean()
    .shift(1)      # Leakage prevention
    .fillna(0)
)

home_gf_form, home_sot_form = [], []
away_gf_form, away_sot_form = [], []

for _, g in team_log.groupby("Team"):
    home_gf_form.append(venue_expanding_mean(g, "H", "GF"))
    home_sot_form.append(venue_expanding_mean(g, "H", "SOT_F"))
    away_gf_form.append(venue_expanding_mean(g, "A", "GF"))
    away_sot_form.append(venue_expanding_mean(g, "A", "SOT_F"))

team_log["Home_GF_Form"] = pd.concat(home_gf_form)
team_log["Home_SOT_Form"] = pd.concat(home_sot_form)
team_log["Away_GF_Form"] = pd.concat(away_gf_form)
team_log["Away_SOT_Form"] = pd.concat(away_sot_form)

# 8. FEATURE 4 — GEOGRAPHICAL FATIGUE

def compute_travel_fatigue(team_name: str, group_df: pd.DataFrame) -> pd.Series:
    """
    Returns a per-match series of cumulative travel miles over
    the team's last 5 games (leakage-free, shift=1).
    """
    if team_name not in STADIUM_COORDS:
        return pd.Series(0.0, index=group_df.index)

    hlat, hlon = STADIUM_COORDS[team_name]
    prev_lat, prev_lon = hlat, hlon  # Season starts at home ground
    distances = []

    for _, row in group_df.iterrows():
        opponent = row["Opponent"]
        if row["Venue"] == "H":
            curr_lat, curr_lon = hlat, hlon
        else:
            curr_lat, curr_lon = STADIUM_COORDS.get(opponent, (hlat, hlon))

```

```

leg_miles = haversine_miles(prev_lat, prev_lon, curr_lat, curr_lon)
distances.append(leg_miles)
prev_lat, prev_lon = curr_lat, curr_lon

dist_series = pd.Series(distances, index=group_df.index)
return dist_series.shift(1).rolling(WINDOW, min_periods=1).sum().fillna(0)

fatigue_parts = []
for team_name, g in team_log.groupby("Team"):
    fatigue_parts.append(compute_travel_fatigue(team_name, g))

team_log["Travel_Fatigue_5"] = pd.concat(fatigue_parts)

```

9. MERGE FEATURES BACK TO MATCH LEVEL

```

FEAT_COLS = [
    "Date", "Team",
    "Roll_GF_5", "Roll_GA_5", "Roll_SOT_5",
    "Home_GF_Form", "Home_SOT_Form",
    "Away_GF_Form", "Away_SOT_Form",
    "Roll_Pts_5", "Travel_Fatigue_5",
]

rename_h = {c: f"H_{c}" for c in FEAT_COLS if c not in ("Date", "Team")}
rename_a = {c: f"A_{c}" for c in FEAT_COLS if c not in ("Date", "Team")}

home_feats = (team_log[team_log["Venue"] == "H"][FEAT_COLS]
               .rename(columns={"Team": "HomeTeam", **rename_h}))
away_feats = (team_log[team_log["Venue"] == "A"][FEAT_COLS]
               .rename(columns={"Team": "AwayTeam", **rename_a}))

df_feats = df.copy()
df_feats = df_feats.merge(home_feats, on=["Date", "HomeTeam"], how="left")
df_feats = df_feats.merge(away_feats, on=["Date", "AwayTeam"], how="left")

```

10. DISPLAY OUTPUTS

```

SHOW_COLS = [
    "Date", "HomeTeam", "AwayTeam", "FTHG", "FTAG", "FTR",
    #           —           Home           team           features
    "H_Roll_GF_5",    # Overall 5-game avg goals scored (home team)
    "H_Roll_SOT_5",   # Overall 5-game avg shots on target (home team)
    "H_Home_GF_Form", # Venue-specific: avg goals at home (home team)

```

```

"H_Home_SOT_Form", # Venue-specific: avg SOT at home (home team)
"H_Roll_Pts_5",    # 5-game rolling points momentum (home team)
"H_Travel_Fatigue_5", # Cumulative travel miles, last 5 games (home team)
#           ————— Away team features
]

"A_Roll_GF_5",
"A_Roll_SOT_5",
"A_Away_GF_Form", # Venue-specific: avg goals away (away team)
"A_Away_SOT_Form",
"A_Roll_Pts_5",
"A_Travel_Fatigue_5",
]

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 220)
pd.set_option("display.float_format", "{:.3f}".format)

print("SECTION 1 — FIRST 10 MATCHES ")
print(df_feats[SHOW_COLS].head(10).to_string(index=False))

print("SECTION 2 — FEATURE SUMMARY STATISTICS")
NUM_COLS = [c for c in SHOW_COLS if c not in
("Date", "HomeTeam", "AwayTeam", "FTR", "FTHG", "FTAG")]
print(df_feats[NUM_COLS].describe().round(3).to_string())

print("SECTION 3 — VENUE FORM DELTA (avg home GF form minus avg away
GF form)")
df_feats["H_Venue_Delta"] = df_feats["H_Home_GF_Form"] -
df_feats["H_Away_GF_Form"]
venue_delta = (
    df_feats.groupby("HomeTeam")["H_Venue_Delta"]
    .mean()
    .sort_values(ascending=False)
    .rename("Home_GF_minus_Away_GF")
    .round(3)
)
print(venue_delta.to_string())

print("SECTION 4 — TRAVEL FATIGUE LEADERS (home team, top 5 most
fatigued)")
print(df_feats[["Date", "HomeTeam", "AwayTeam", "H_Travel_Fatigue_5"]]
.sort_values("H_Travel_Fatigue_5", ascending=False))

```

```

        .head(5).to_string(index=False))

print("SECTION 5 — TRAVEL FATIGUE LEADERS (away team, top 5 most
fatigued)")
print(df_feats[["Date","HomeTeam","AwayTeam","A_Travel_Fatigue_5"]]
      .sort_values("A_Travel_Fatigue_5", ascending=False)
      .head(5).to_string(index=False))

print("SECTION 6 — ROLLING MOMENTUM RANKINGS (avg 5-game pts, full
season)")
h_pts = df_feats.groupby("HomeTeam")["H_Roll_Pts_5"].mean()
a_pts = df_feats.groupby("AwayTeam")["A_Roll_Pts_5"].mean()
momentum = ((h_pts + a_pts) / 2).sort_values(ascending=False).round(3)
print(momentum.to_string())

# Save engineered feature matrix
output_path = "epl_features_2324.csv"
df_feats[SHOW_COLS].to_csv(output_path, index=False)
print(f"\nFeature matrix saved → {output_path}")

import pandas as pd
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import TimeSeriesSplit
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import log_loss
import warnings
warnings.filterwarnings("ignore")
# Load engineered feature matrix
df_model = pd.read_csv("epl_features_2324.csv")

# Define features and target
features = [
    "H_Roll_GF_5",          "H_Roll_SOT_5",          "H_Home_GF_Form",
    "H_Home_SOT_Form", "H_Roll_Pts_5", "H_Travel_Fatigue_5",
    "A_Roll_GF_5",          "A_Roll_SOT_5",          "A_Away_GF_Form",
    "A_Away_SOT_Form", "A_Roll_Pts_5", "A_Travel_Fatigue_5"
]
X = df_model[features]
y = df_model["FTR"]

# 1. Time-Series Cross-Validation (5 Folds)
tscv = TimeSeriesSplit(n_splits=5)
cv_log_losses = []

```

```

# 2. Walk-forward Validation Loop
for train_index, test_index in tscv.split(X):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    # Scale features (Crucial for Logistic Regression)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Fit Logistic Regression with moderate regularization (C=0.1)
    model = LogisticRegression(multi_class='multinomial', solver='lbfgs', C=0.1,
max_iter=1000)
    model.fit(X_train_scaled, y_train)

    # Predict probabilities
    probs = model.predict_proba(X_test_scaled)
    loss = log_loss(y_test, probs, labels=['A', 'D', 'H'])
    cv_log_losses.append(loss)

print(f'Logistic Regression CV Log-Loss: {np.mean(cv_log_losses):.4f}')

# Train final model on all data to see coefficients
scaler = StandardScaler()
X_final_scaled = scaler.fit_transform(X)
model.fit(X_final_scaled, y)

# Show coefficients for Home Win (H)
coef_df = pd.DataFrame(model.coef_[2], index=features, columns=['Coefficient'])
print("\n--- Impact on Home Win (H) ---")
print(coef_df.sort_values(by='Coefficient', ascending=False))
df_market = df[['FTR', 'B365H', 'B365D', 'B365A']].copy()

# 1. Convert decimal odds to implied probabilities
df_market['implied_h'] = 1 / df_market['B365H']
df_market['implied_d'] = 1 / df_market['B365D']
df_market['implied_a'] = 1 / df_market['B365A']

# 2. Normalize so probabilities sum to 1.0 (remove overround)
df_market['sum_implied'] = df_market['implied_h'] + df_market['implied_d'] +
df_market['implied_a']
df_market['prob_h'] = df_market['implied_h'] / df_market['sum_implied']
df_market['prob_d'] = df_market['implied_d'] / df_market['sum_implied']
df_market['prob_a'] = df_market['implied_a'] / df_market['sum_implied']

```

```
# 3. Calculate Market Log-Loss
market_probs = df_market[['prob_a', 'prob_d', 'prob_h']].values # Order: A, D, H
market_loss = log_loss(df_market['FTR'], market_probs, labels=['A', 'D', 'H'])

print(f"\nBet365 Market Baseline Log-Loss: {market_loss:.4f}")
```